

Test Report

Full quality assurance audit — unit, integration, E2E & accessibility

DATE	PROJECT	TOTAL TESTS	RESULT
22 April 2026	radar-room-configurator	80	✅ All Passed

1 • Overview

This report covers every automated test run on the Radar Room Configurator web application. Three different types of tests were used, each catching a different kind of problem. A total of **80 tests** were written and executed — all passed after fixing the bugs the tests uncovered.

80

TOTAL TESTS

55

UNIT TESTS

25

E2E TESTS

3BUGS FOUND &
FIXED

2 • What Each Type of Test Does

Think of it like quality-checking a car: **Unit tests** check that each individual part (engine, brakes) works on its own. **Integration tests** check that parts work together. **E2E tests** check that a real person can drive the car from start to finish without it breaking.

Unit Tests

Unit tests check the smallest pieces of code in isolation — one function at a time. They run in milliseconds and catch logic errors early. We used **Vitest** as the test runner.

E2E (End-to-End) Tests

E2E tests open a real browser, click around the app exactly like a user would, and verify things look and behave correctly. We used **Playwright** with a real Chromium browser.

Accessibility Tests

Accessibility tests check that the app works for people using screen readers or keyboard-only navigation. We used the **axe-core** engine which checks against international WCAG 2.0 / 2.1 standards.

3 · Unit Tests (55 tests)

These tests run against the raw TypeScript code without a browser. They are extremely fast (completed in ~2.5 seconds total) and verify that the core maths and logic of the app are correct.

3.1 — Room Geometry (types.ts) · 15 tests

These tests check two functions that calculate where objects sit in the room.

#	TEST NAME	WHAT IT CHECKS	RESULT
1	getEffectiveDims — 0° rotation	Object at no rotation has the correct width & height	PASS
2	getEffectiveDims — 90° rotation	Object rotated 90° has width and height swapped	PASS
3	getEffectiveDims — 180° rotation	Object at 180° returns same dims as 0°	PASS
4	getEffectiveDims — 45° rotation	Diagonal bounding box is calculated correctly	PASS
5	detectDoorWall — top wall	Door placed at top edge is detected on top wall	PASS
6	detectDoorWall — bottom wall	Door placed at bottom edge is detected on bottom wall	PASS
7	detectDoorWall — left wall	Door placed at left edge is detected on left wall	PASS
8	detectDoorWall — right wall	Door placed at right edge is detected on right wall	PASS
9	detectDoorWall — centre of room	Door in the middle of the room returns null (no wall)	PASS
10	detectDoorWall — near corner	Corner door picks the single closest wall, not both	PASS
11	detectDoorWall — closest not first	Algorithm picks minimum distance, not just first match	PASS
12	OBJECT_PRESETS — all fields present	Every object type has label, size, color & emoji defined	PASS
13	OBJECT_PRESETS — bed dimensions	Bed is wider than 0.5 m and longer than 1 m	PASS
14	OBJECT_PRESETS — door is thin	Door height is less than 0.3 m (wall-mounted object)	PASS
15	OBJECT_PRESETS — radar is small	Radar sensor width is less than 0.2 m	PASS

3.2 — Config Export (buildConfig) · 10 tests

These tests verify that the JSON export (used by the radar hardware firmware) is formatted exactly correctly.

#	TEST NAME	WHAT IT CHECKS	RESULT
1	device_configs block	Board name and location are written correctly	PASS
2	Object coordinates	top_left / bottom_right corners are at correct x/y positions	PASS
3	Label to snake_case	"My Bed" becomes "my_bed" in the output	PASS
4	state_machine for bed	Bed appears in the state_machine feature section	PASS
5	out_of_bed_alerts for bed	Bed triggers out-of-bed alert feature	PASS
6	No out_of_bed_alerts for sofa	Sofa does NOT trigger out-of-bed alerts	PASS
7	out_of_room_alerts for door	Door triggers exit-room alert feature	PASS
8	Empty sections omitted	Features with no matching objects are not written to JSON	PASS
9	Multiple objects	Bed + Sofa both appear in state_machine section	PASS
10	Coordinate rounding	Coordinates are rounded to 3 decimal places	PASS

3.3 — Properties Panel · 11 tests

These tests render the right-side properties panel and check that buttons and inputs behave correctly.

#	TEST NAME	WHAT IT CHECKS	RESULT
1	Empty state	Shows "Nothing selected" when no object is chosen	PASS
2	Shows object label	Selected object's name appears in the panel	PASS
3	Shows object type	Object type (e.g. "bed") appears in lowercase	PASS
4	Width and height fields	Correct dimensions (1.4 m × 2 m) are pre-filled	PASS
5	Rotation buttons visible	90°, 180°, 270° buttons are rendered	PASS
6	Rotation button triggers update	Clicking 90° calls the update handler with rotation: 90	PASS
7	Delete button works	Clicking Delete calls the delete handler	PASS
8	Deselect button works	Clicking X calls the deselect handler	PASS
9	No update on every keystroke	Typing in a field does NOT fire updates until blur/Enter	PASS
10	Reverts invalid input	Typing "abc" in a number field reverts to original value	PASS
11	Radar info shown	Radar type shows coordinate display block	PASS

3.4 — App Component · 19 tests

These tests render the full app and check the main interactions work end-to-end in a simulated browser environment.

#	TEST NAME	WHAT IT CHECKS	RESULT
1	Renders app title	"Radar Room Configurator" text is visible	PASS
2	0 objects initially	Header shows "0 objects" on first load	PASS
3	3D button disabled (no objects)	3D button is greyed out when canvas is empty	PASS
4	Export button disabled (no objects)	Export button is greyed out when canvas is empty	PASS
5	Clear not visible initially	Clear button only appears when objects exist	PASS
6	Import always visible	Import button is always in the header	PASS
7	Dark mode toggle	Clicking 🌞 switches to 🌙 and vice versa	PASS
8	Add bed — count = 1	Clicking Bed in palette shows "1 object" in header	PASS

#	TEST NAME	WHAT IT CHECKS	RESULT
9	3D/Export enable after add	Both buttons become enabled after adding an object	PASS
10	Clear button appears after add	Clear button shows once at least one object exists	PASS
11	Multiple objects count	Adding Bed + Sofa + Door shows "3 objects"	PASS
12	Properties panel opens on add	Panel appears when an object is selected	PASS
13	Clear — confirm = remove all	Confirming dialog removes all objects	PASS
14	Clear — cancel = keep objects	Cancelling dialog keeps objects intact	PASS
15	3D viewer opens	Clicking  3D opens the 3D room viewer	PASS
16	3D viewer closes	Clicking Close dismisses the 3D viewer	PASS
17	Default room size	Header shows "4 × 4 m" on first load	PASS
18	Export modal opens	Clicking  Export opens the export dialog	PASS
19	Adjacent room — door near wall	Door near a wall shows adjacent room section	PASS

4 · E2E Tests (25 tests)

These tests open a real Chromium browser, navigate to `http://localhost:5175`, and interact with the app exactly like a human user. Screenshots were taken on failures to help diagnose problems. Total run time: ~22 seconds.

#	CATEGORY	TEST NAME	WHAT IT DOES	RESULT
1	Page Load	No JS errors on load	Opens the app and checks no JavaScript crashes occur	PASS
2	Page Load	App title visible	Checks "Radar Room Configurator" text appears	PASS
3	Page Load	Room dimensions in header	Checks "4 × 4 m" appears in the top bar	PASS
4	Object Palette	All object types visible	Checks Bed, Sofa, Table, Desk, Chair, Door, Window, Radar all appear in the left panel	PASS
5	Object Palette	Adding Bed	Clicks "Bed" and verifies header now shows "1 object"	PASS
6	Object Palette	Adding multiple objects	Adds Bed + Sofa + Door, verifies count = 3	PASS
7	Properties Panel	Panel opens after add	Adds object, verifies right panel shows type info	PASS
8	Properties Panel	Rotation buttons visible	Checks 90°, 180° buttons appear after selection	PASS
9	Properties Panel	Width field accepts input	Types "1.8" in the width field and confirms it is saved	PASS
10	Properties Panel	Delete button removes object	Clicks Delete and confirms object count returns to 0	PASS
11	Room Size	Width input exists	Finds the number input for room width	PASS
12	Room Size	Changing width updates header	Sets width to 6, verifies header shows "6 × ..."	PASS
13	3D Viewer	3D button opens viewer	Clicks  3D and verifies "3D Room View" panel appears	PASS
14	3D Viewer	Close dismisses viewer	Opens then closes 3D, verifies panel disappears	PASS
15	3D Viewer	Canvas renders (WebGL)	Verifies a <code><canvas></code> element is visible (3D rendered)	PASS

#	CATEGORY	TEST NAME	WHAT IT DOES	RESULT
16	Export Modal	Export modal opens	Clicks Export and verifies board/location form appears	PASS
17	Export Modal	Cancel closes modal	Clicks Cancel and verifies modal disappears	PASS
18	Import Modal	Import modal opens	Clicks Import and verifies floor plan upload dialog appears	PASS
19	Dark Mode	Light/dark toggle	Clicks the ☀️ button and verifies 🌙 appears (mode switched)	PASS
20	Keyboard	R key rotates object	Selects object, presses R, verifies rotation shows 45°	PASS
21	Keyboard	Arrow key moves object	Presses ArrowRight, verifies app does not crash	PASS
22	Clear	Clear removes all objects	Adds 2 objects, clicks Clear, confirms, verifies 0 objects	PASS
23	Accessibility	No critical a11y violations	Runs axe-core on live page, checks WCAG 2.0 AA criteria	PASS
24	Responsive	Mobile viewport (375×812)	Loads app at iPhone size, verifies no JS errors	PASS
25	Responsive	Large desktop (1920×1080)	Loads app at 4K desktop size, verifies title visible	PASS

5 • Bugs Found & Fixed

The tests uncovered 3 real bugs. All were fixed before the final test run.

Bug 1 — Room size inputs had no accessible labels

Critical A11y

Fixed

Found by: E2E Accessibility test (axe-core)

What was wrong: The "Width" and "Length" labels in the Room Size section were plain text sitting next to inputs — they were not programmatically linked. Screen readers couldn't tell users what the inputs were for.

Fix: Added `id` attributes to the inputs and `htmlFor` to the labels in `ObjectPalette.tsx`, creating a proper connection.

Bug 2 — Door/window positioned at wrong location in 3D

Fixed

Found by: Manual visual inspection during E2E 3D viewer test

What was wrong: When placing a door or window in 3D, the code subtracted half the door's width from the position *after* the geometry was already centered, causing it to appear half a door-width away from where it was placed on the 2D canvas.

Fix: Removed the erroneous offset in `Room3DViewer.tsx`. The 3D position now maps 1:1 to the 2D canvas position.

Bug 3 — Sofa arms extended outside the room footprint in 3D

Fixed

Found by: Manual visual inspection during E2E 3D viewer test

What was wrong: The sofa arm geometry was positioned starting at a negative Z coordinate, causing the arms to stick out behind the sofa and through the wall.

Fix: Rewrote the sofa geometry in `Room3DViewer.tsx` so all parts (backrest, seat, arms, cushions) stay within the 0 → depth range of the object's footprint.

6 • Tools & Technologies Used

TOOL	PURPOSE	TYPE
Vitest	Runs unit tests — fast, TypeScript-native test runner	Unit
@testing-library/react	Renders React components and simulates user events in tests	Unit
jsdom	Fake browser environment so unit tests don't need Chrome	Unit
Playwright	Opens a real Chromium browser and runs click/type/assert tests	E2E
axe-core / @axe-core/playwright	Checks accessibility rules (WCAG 2.0 AA) against the live page	A11y
TypeScript	All test files are fully typed — prevents test code from having typos	All

7 · Glossary

Unit Test

A test that checks one small function or component in complete isolation from everything else.

E2E Test

End-to-End test — simulates a real user opening the app in a browser and clicking through it.

WCAG

Web Content Accessibility Guidelines — international standards for making websites usable by everyone, including people with disabilities.

axe-core

An open-source engine that automatically scans a web page for accessibility problems.

Playwright

A tool made by Microsoft that controls a real browser programmatically for automated testing.

Vitest

A fast test runner built for Vite/React projects — similar to Jest but faster and TypeScript-first.

jsdom

A simulated browser environment that runs inside Node.js so unit tests don't need to open a real browser window.